

Week 2: MLFQ Scheduler Implementation

Core Scheduler Logic and Demotion Policy

Abdullah Khan Sherwani 27880

Muhammad Anas Tabba 28998

3rd December 2025

Contents

1	Overview	3
1.1	Week 2 Objectives	3
2	Scheduler Implementation	3
2.1	MLFQ Scheduler Logic	3
2.2	Priority Boosting Function	4
3	Time Slice Tracking and Demotion	5
3.1	Timer Interrupt Handler	5
3.2	Demotion Policy	6
4	Test Programs	6
4.1	CPU-Bound Test (cpubound.c)	6
4.2	I/O-Bound Test (iobound.c)	7
4.3	Mixed Workload Test (mlfqtest.c)	8
5	Testing Results	9
5.1	Build and Run	9
5.2	CPU-Bound Test Output	10
5.3	I/O-Bound Test Output	10
5.4	Mixed Workload Test Output	10
6	MLFQ Behavior Analysis	11
6.1	CPU-Bound Process Behavior	11
6.2	I/O-Bound Process Behavior	11
6.3	Key Insight: Voluntary Yield	11
7	Files Modified in Week 2	12
7.1	Kernel Files	12
7.2	User Test Programs	12
7.3	Build System	12

8	Week 2 Summary	12
8.1	Completed Tasks	12
8.2	Validation Results	12
8.3	Next Steps (Week 3)	12

1 Overview

This document describes the core implementation of the Multi-Level Feedback Queue (MLFQ) scheduler in xv6-RISC-V. Building on the design from Week 1, Week 2 focuses on implementing the actual scheduling logic, time slice tracking, and automatic demotion policy.

1.1 Week 2 Objectives

1. Implement priority-based MLFQ scheduling in `scheduler()`
2. Add per-tick time slice tracking in the timer interrupt handler
3. Implement automatic demotion when time quantum is exhausted
4. Create comprehensive test programs for CPU-bound and I/O-bound workloads
5. Validate scheduler behavior through testing

2 Scheduler Implementation

2.1 MLFQ Scheduler Logic

The `scheduler()` function in `kernel/proc.c` was modified to implement priority-based scheduling:

```
1 void
2 scheduler(void)
3 {
4     struct proc *p;
5     struct cpu *c = mycpu();
6
7     c->proc = 0;
8     for(;;){
9         intr_on();
10        intr_off();
11
12        // Check if it's time for priority boosting (starvation
13        // prevention)
14        acquire(&tickslock);
15        uint64 current_ticks = ticks;
16        if(current_ticks - last_boost_time >= BOOST_INTERVAL) {
17            last_boost_time = current_ticks;
18            release(&tickslock);
19            boost_all_priorities();
20        } else {
21            release(&tickslock);
22        }
23
24        int found = 0;
```

```

25 // MLFQ Scheduling: Scan priority queues from highest (0) to
    lowest (3)
26 for(int priority = 0; priority < NMLFQ && !found; priority++)
    {
27     for(p = proc; p < &proc[NPROC]; p++) {
28         acquire(&p->lock);
29         if(p->state == RUNNABLE && p->priority == priority) {
30             // Switch to chosen process
31             p->state = RUNNING;
32             c->proc = p;
33             swtch(&c->context, &p->context);
34
35             // Process is done running for now
36             c->proc = 0;
37             found = 1;
38             release(&p->lock);
39             break; // Restart from highest priority
40         }
41         release(&p->lock);
42     }
43 }
44
45 if(found == 0) {
46     // Nothing to run; wait for interrupt
47     asm volatile("wfi");
48 }
49 }
50 }

```

Listing 1: MLFQ Scheduler Implementation (kernel/proc.c)

Key Algorithm:

1. Check for priority boost opportunity (every 30 ticks)
2. Scan queues from Q0 (highest) to Q3 (lowest)
3. Select the first RUNNABLE process from the highest non-empty queue
4. After running a process, restart scan from Q0

2.2 Priority Boosting Function

To prevent starvation, all processes are periodically boosted to Q0:

```

1 void
2 boost_all_priorities(void)
3 {
4     struct proc *p;
5     int boosted_count = 0;
6
7     for(p = proc; p < &proc[NPROC]; p++) {
8         acquire(&p->lock);

```

```

9      if(p->state == RUNNABLE || p->state == RUNNING || p->state ==
      SLEEPING) {
10          if(p->priority != 0 || p->time_slices != 0) {
11              boosted_count++;
12          }
13          p->priority = 0;
14          p->time_slices = 0;
15      }
16      release(&p->lock);
17  }
18 }

```

Listing 2: Priority Boost Function (kernel/proc.c)

3 Time Slice Tracking and Demotion

3.1 Timer Interrupt Handler

Modified usertrap() in kernel/trap.c to track time slices and handle demotions:

```

1  uint64
2  usertrap(void)
3  {
4      int which_dev = 0;
5      // ... existing trap handling code ...
6
7      // give up the CPU if this is a timer interrupt.
8      if(which_dev == 2) {
9          // MLFQ: Track time slices and handle demotion
10         p->time_slices++;
11
12         // Check if process has used its time quantum for current
13         priority
14         extern int mlfq_time_quanta[];
15         #define NMLFQ 4
16
17         if(p->time_slices >= mlfq_time_quanta[p->priority]) {
18             // Process used full quantum, demote to lower priority
19             queue
20             if(p->priority < NMLFQ - 1) {
21                 p->priority++; // Move to lower priority queue
22             }
23             p->time_slices = 0; // Reset time slice counter for new
24             queue
25         }
26
27         yield();
28     }
29
30     // ... return to user space ...
31 }

```

Listing 3: Time Tracking and Demotion (kernel/trap.c)

3.2 Demotion Policy

The demotion thresholds based on time quanta:

Queue	Quantum	Demotion After	Next Queue
Q0	2 ticks	2 ticks used	Q1
Q1	4 ticks	4 ticks used	Q2
Q2	8 ticks	8 ticks used	Q3
Q3	16 ticks	N/A (stays)	Q3

Table 1: Demotion Thresholds

4 Test Programs

4.1 CPU-Bound Test (cpubound.c)

Tests that long-running computational processes are demoted:

```

1 #include "kernel/types.h"
2 #include "kernel/stat.h"
3 #include "user/user.h"
4
5 int
6 main(int argc, char *argv[])
7 {
8     struct procinfo info;
9     volatile int dummy = 0;
10
11     printf("=== Comprehensive CPU-Bound Test ===\n");
12     printf("Watch priority change from Q0 -> Q1 -> Q2 -> Q3\n\n");
13
14     if(getprocinfo(&info) == 0) {
15         printf("Starting: PID=%d, Priority=Q%d, TimeSlices=%d\n\n",
16             info.pid, info.priority, info.time_slices);
17     }
18
19     // Massive continuous CPU work without any yields
20     for(int iter = 0; iter < 50; iter++) {
21         for(long j = 0; j < 10000000; j++) {
22             dummy = dummy + j;
23             dummy = dummy % 1000000;
24         }
25
26         if(getprocinfo(&info) == 0) {
27             printf("Checkpoint %d: Priority=Q%d, TimeSlices=%d\n",
28                 iter, info.priority, info.time_slices);

```

```

29     }
30 }
31
32 if(getprocinfo(&info) == 0) {
33     printf("\nFinal Result: Priority=Q%d\n", info.priority);
34     if(info.priority == 3) {
35         printf("SUCCESS: Reached Q3 (CPU-bound category)\n");
36     }
37 }
38
39 exit(0);
40 }

```

Listing 4: CPU-Bound Test Program (user/cpubound.c)

4.2 I/O-Bound Test (iobound.c)

Tests that processes yielding frequently maintain high priority:

```

1 #include "kernel/types.h"
2 #include "kernel/stat.h"
3 #include "user/user.h"
4
5 int
6 main(int argc, char *argv[])
7 {
8     struct procinfo info;
9
10    printf("=== Comprehensive I/O-Bound Test ===\n");
11    printf("Should STAY in Q0 because it yields frequently.\n\n");
12
13    if(getprocinfo(&info) == 0) {
14        printf("Starting: PID=%d, Priority=Q%d\n\n",
15              info.pid, info.priority);
16    }
17
18    // Simulate I/O-bound behavior: short bursts with frequent
19    // sleeps
20    for(int iter = 0; iter < 30; iter++) {
21        volatile int dummy = 0;
22        for(long j = 0; j < 2000000; j++) {
23            dummy = dummy + j;
24        }
25
26        pause(1); // Sleep for 1 tick - yields before quantum
27        // exhausted
28
29        if(iter % 5 == 0 && getprocinfo(&info) == 0) {
30            printf("Iteration %d: Priority=Q%d, TimeSlices=%d\n",
31                  iter, info.priority, info.time_slices);
32        }
33    }
34 }

```

```

32
33     if(getprocinfo(&info) == 0) {
34         printf("\nFinal Result: Priority=Q%d\n", info.priority);
35         if(info.priority <= 1) {
36             printf("SUCCESS: Stayed in high priority!\n");
37         }
38     }
39
40     exit(0);
41 }

```

Listing 5: I/O-Bound Test Program (user/iobound.c)

4.3 Mixed Workload Test (mlfqtest.c)

Runs CPU-bound and I/O-bound processes concurrently:

```

1 #include "kernel/types.h"
2 #include "kernel/stat.h"
3 #include "user/user.h"
4
5 int
6 main(int argc, char *argv[])
7 {
8     int cpid1, cpid2;
9
10    printf("=== Mixed Workload Test ===\n");
11    printf("CPU-bound vs I/O-bound concurrently\n\n");
12
13    // Fork CPU-bound process
14    cpid1 = fork();
15    if(cpid1 == 0) {
16        printf("[CPU-BOUND] Starting...\n");
17        volatile int dummy = 0;
18
19        for(int iter = 0; iter < 40; iter++) {
20            for(long j = 0; j < 10000000; j++) {
21                dummy = dummy + j;
22                dummy = dummy % 1000000;
23            }
24
25            struct procinfo info;
26            if(iter % 10 == 0 && getprocinfo(&info) == 0) {
27                printf("[CPU-BOUND] Iter %d: Q%d\n", iter, info.priority)
28            ;
29            }
30
31            struct procinfo final;
32            if(getprocinfo(&final) == 0) {
33                printf("[CPU-BOUND] Final: Q%d\n", final.priority);
34            }

```



```
35     exit(0);
36 }
37
38 pause(1);
39
40 // Fork I/O-bound process
41 cpid2 = fork();
42 if(cpid2 == 0) {
43     printf("[I/O-BOUND] Starting...\n");
44
45     for(int iter = 0; iter < 25; iter++) {
46         volatile int dummy = 0;
47         for(long j = 0; j < 2000000; j++) {
48             dummy = dummy + j;
49         }
50
51         pause(1);
52
53         struct procinfo info;
54         if(iter % 5 == 0 && getprocinfo(&info) == 0) {
55             printf("[I/O-BOUND] Iter %d: Q%d\n", iter, info.priority)
56         ;
57         }
58
59         struct procinfo final;
60         if(getprocinfo(&final) == 0) {
61             printf("[I/O-BOUND] Final: Q%d\n", final.priority);
62         }
63         exit(0);
64     }
65
66     wait(0);
67     wait(0);
68
69     printf("\n=== Expected Results ===\n");
70     printf("CPU-bound: Demoted to Q2 or Q3\n");
71     printf("I/O-bound: Stayed in Q0 or Q1\n");
72
73     exit(0);
74 }
```

Listing 6: Mixed Workload Test (user/mlfqtest.c)

5 Testing Results

5.1 Build and Run

```
1 $ make clean && make
2 $ make qemu CPUS=1
```

Listing 7: Build Commands

5.2 CPU-Bound Test Output

```
1 $ cpubound
2 === Comprehensive CPU-Bound Test ===
3 Watch priority change from Q0 -> Q1 -> Q2 -> Q3
4
5 Starting: PID=3, Priority=Q0, TimeSlices=0
6
7 Checkpoint 0: Priority=Q0, TimeSlices=1
8 Checkpoint 1: Priority=Q1, TimeSlices=3
9 Checkpoint 2: Priority=Q2, TimeSlices=5
10 Checkpoint 3: Priority=Q3, TimeSlices=9
11 Checkpoint 4: Priority=Q3, TimeSlices=14
12 ...
13
14 Final Result: Priority=Q3
15 SUCCESS: Reached Q3 (CPU-bound category)
```

Listing 8: cpubound Output

5.3 I/O-Bound Test Output

```
1 $ iobound
2 === Comprehensive I/O-Bound Test ===
3 Should STAY in Q0 because it yields frequently.
4
5 Starting: PID=3, Priority=Q0
6
7 Iteration 0: Priority=Q0, TimeSlices=0
8 Iteration 5: Priority=Q0, TimeSlices=1
9 Iteration 10: Priority=Q0, TimeSlices=0
10 Iteration 15: Priority=Q0, TimeSlices=1
11 Iteration 20: Priority=Q0, TimeSlices=0
12 Iteration 25: Priority=Q0, TimeSlices=1
13
14 Final Result: Priority=Q0
15 SUCCESS: Stayed in high priority!
```

Listing 9: iobound Output

5.4 Mixed Workload Test Output

```
1 $ mlfqtest
2 === Mixed Workload Test ===
3 CPU-bound vs I/O-bound concurrently
```

```

4
5 [CPU-BOUND] Starting...
6 [I/O-BOUND] Starting...
7 [I/O-BOUND] Iter 0: Q0
8 [CPU-BOUND] Iter 0: Q0
9 [I/O-BOUND] Iter 5: Q0
10 [CPU-BOUND] Iter 10: Q2
11 [I/O-BOUND] Iter 10: Q0
12 [CPU-BOUND] Iter 20: Q3
13 [I/O-BOUND] Iter 15: Q0
14 [I/O-BOUND] Iter 20: Q0
15 [I/O-BOUND] Final: Q0
16 [CPU-BOUND] Iter 30: Q3
17 [CPU-BOUND] Final: Q3
18
19 === Expected Results ===
20 CPU-bound: Demoted to Q2 or Q3
21 I/O-bound: Stayed in Q0 or Q1

```

Listing 10: mlfqtest Output

6 MLFQ Behavior Analysis

6.1 CPU-Bound Process Behavior

1. Process starts at Q0 with 2-tick quantum
2. Uses all 2 ticks → demoted to Q1
3. Uses all 4 ticks in Q1 → demoted to Q2
4. Uses all 8 ticks in Q2 → demoted to Q3
5. Stays at Q3 with 16-tick quantum until boost

6.2 I/O-Bound Process Behavior

1. Process starts at Q0 with 2-tick quantum
2. Does brief work (1 tick), then sleeps (yields)
3. Never uses full quantum → stays in Q0
4. Maintains high priority for responsiveness

6.3 Key Insight: Voluntary Yield

The key to I/O-bound behavior is **voluntary yielding**:

- When a process calls `sleep()` or `pause()`, it yields the CPU
- The process keeps its current `time_slices` count

- Only when the full quantum is exhausted does demotion occur
- This naturally favors interactive/I/O-bound processes

7 Files Modified in Week 2

7.1 Kernel Files

- `kernel/proc.c` – Implemented MLFQ scheduler logic and boost function
- `kernel/trap.c` – Added time slice tracking and demotion in timer handler

7.2 User Test Programs

- `user/cpubound.c` – CPU-intensive test
- `user/iobound.c` – I/O-intensive test
- `user/mlfqtest.c` – Mixed workload test

7.3 Build System

- `Makefile` – Added new test programs to UPROGS

8 Week 2 Summary

8.1 Completed Tasks

Implemented priority-based MLFQ scheduling in `scheduler()`

Added per-tick time slice tracking in timer interrupt handler

Implemented automatic demotion policy

Implemented priority boosting for starvation prevention

Created and validated CPU-bound test

Created and validated I/O-bound test

Created and validated mixed workload test

8.2 Validation Results

8.3 Next Steps (Week 3)

1. Fine-tune boost interval if needed
2. Add `boostproc()` syscall for manual testing
3. Create extended boost demonstration test

Test	Expected	Result
cpubound	Demotes to Q3	PASS
iobound	Stays in Q0	PASS
mlfqtest	I/O gets priority	PASS
procinfo	Returns correct data	PASS

Table 2: Week 2 Test Results

4. Comprehensive stress testing
5. Final documentation and report